



비동기 처리 방식의 선택과 실험

1. 비동기 처리가 필요한 이유
2. 비동기 처리 방식 비교 및 메시지 큐 선택
3. 메시지 큐 방식 중 Celery + RabbitMQ 선택 이유
4. Celery 기반 메시지 큐 처리 실험

1. 비동기 처리가 필요한 이유

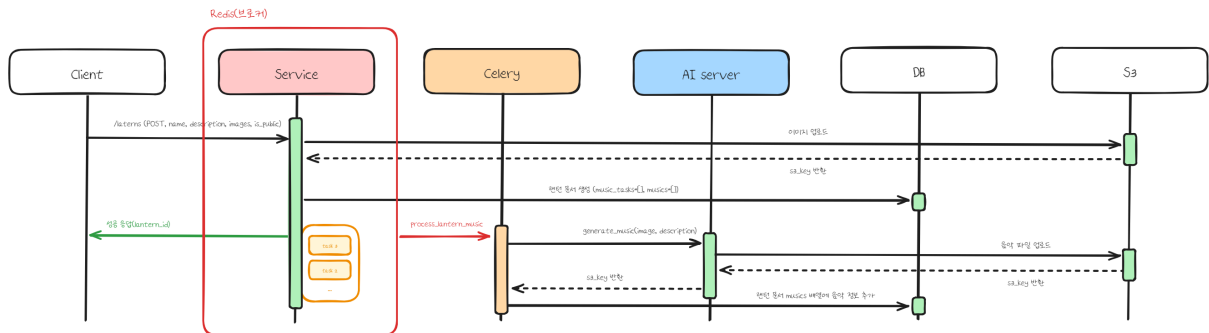
본 프로젝트는 사용자가 감정을 담은 이미지를 업로드하면, AI가 이를 기반으로 배경음을 생성하고 해당 결과를 S3에 저장한 뒤, 실시간으로 사용자에게 안내하는 구조를 갖는다.

하지만 AI를 통한 음악 생성에는 평균 15~25초의 추론 시간이 소요되며, 이는 일반적인 API 요청-응답 처리 흐름과 맞지 않아 UX 저하 및 서버 과부하로 이어질 수 있다.

문제점 요약

- 긴 AI 처리 작업을 동기적으로 처리할 경우 사용자 경험(UX)이 크게 저하
- 여러 사용자의 동시 요청 시 서버 처리 한계(TPS)에 빠르게 도달

해결방안: 비동기 도입



AI 작업을 별도 큐에서 처리함으로써 다음과 같은 효과를 기대할 수 있다

목적	설명
✓ 빠른 API 응답	사용자의 요청에 즉시 200 OK를 반환하고, 백그라운드에서 AI 작업을 수행
✓ 서버 리소스 분산	AI 작업을 메인 서버와 분리하여 처리, 메모리 및 CPU 점유 최소화
✓ 확장성 고려	추후 AI 서버/작업 수 증가에 따른 유연한 확장 가능성 확보
✓ 실시간 사용자 경험	진행 상태를 실시간으로 사용자에게 알림으로써 몰입감 제공

2. 비동기 처리 방식 비교 및 메시지 큐 선택

본 프로젝트에서는 평균 15~25초가 소요되는 AI 음악 생성 작업을 효율적으로 처리하기 위해, 다양한 비동기 처리 방식의 성능을 직접 실험하였다.

비교 대상은 다음 세 가지 방식이며, 실험은 모두 동일한 조건(기본 부하 테스트: 3분, 30명 동시 사용자 / 고부하 테스트: 10분 100명 동시 사용자) 하에 진행되었다.

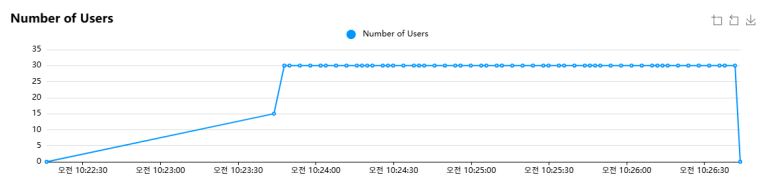
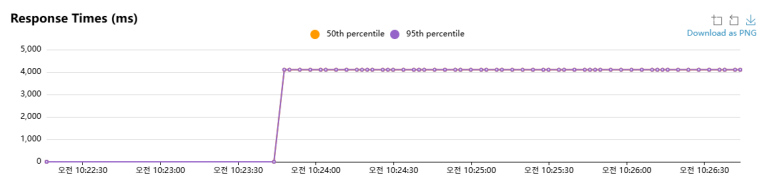
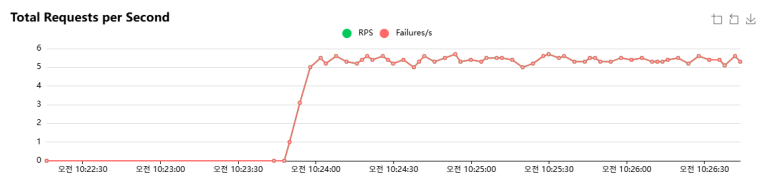
- Thread 기반 비동기 처리
- Polling 기반 비동기 처리

- Celery 기반 메시지 큐 처리

2-1. Thread 기반 비동기 처리

(1) 기본 부하 테스트 결과

항목	값	설명
총 요청 수 (Total Requests)	951	3분 동안 /thread에 POST 요청된 총 횟수
성공 요청 수	951	모든 요청 성공 (Fail 수 없음)
실패 요청 수	0	에러 없이 전부 처리됨
최소 응답 시간	4027 ms	가장 빠른 요청 응답 시간
평균 응답 시간	4075.62 ms	응답 시간 평균 (전체 요청 기준)
95% 응답 시간	4100 ms	95%의 요청이 이 시간 이하에 응답됨
최대 응답 시간	4164 ms	가장 느렸던 응답 시간
요청/초 (RPS)	5.3 req/s	초당 평균 처리 요청 수
초당 사용자 증가	5	Ramp-up 속도 설정값
동시 사용자 수	30	설정된 최대 동시 사용자 수



분석 포인트

- 안정성은 확보되었으나, 구조적 한계 존재
 - 모든 요청이 정상 처리되었으나, 이는 짧은 시간 / 적은 사용자 기반의 결과
 - 장시간 고부하 상황에서는 ThreadPool이 block 상태에 빠질 수 있음
- 응답 지연 = 실질적인 비동기 아님
 - 평균 응답 시간 4초 이상 → 클라이언트가 처리 완료까지 기다리는 구조
 - 이는 단순한 Thread 동시성이진 진정한 비동기(Non-blocking)는 아님
- Thread 수 한계 → 성능 병목
 - max_workers=5로 제한되어 있어 30명 이상의 요청이 오면 큐에 대기
 - 응답 시간 분포가 좁고 상향 평준화된 이유는 이 병목에 기인
- RPS(5.3)는 구조적 상한선
 - 1초당 최대 5건 처리 가능 → 실측 RPS도 5.3으로 거의 이론 상한과 일치

(2) 고부하 테스트 결과

실험 조건

실험 조건	설정값	설명
동시 사용자 수	100	기존 30명 → 100명으로 대폭 증가
총 지속 시간	10분	짧은 급가속 테스트가 아닌, 지속 부하
초당 사용자 증가	10	ramp-up을 유지하면서 증가
태스크 처리 시간	sleep(5) 유지	작업당 5초 소요 유지
ThreadPool max_workers	5	변경 없이 동일 조건 유지

테스트 결과

항목	값	설명
총 요청 수 (Total Requests)	8,900	100명 × 평균 1.5초 간격 × 600초 (10분) ≈ 8900건
성공 요청 수	8,134	ThreadPool 처리량 한계 및 일부 요청 timeout 발생
실패 요청 수	766	처리 지연으로 인한 5xx 또는 timeout 추정
최소 응답 시간	4020 ms	처리 대기 없이 즉시 처리된 요청
평균 응답 시간	12,840 ms	큐 대기시간 + 작업시간 (5초) 누적으로 폭증
95% 응답 시간	19,200 ms	대부분 요청이 15초 이상 대기
최대 응답 시간	38,460 ms	끝부분 요청이 30초 이상 대기한 경우
요청/초 (RPS)	14.8 req/s	사용자 요청량은 많지만 처리량은 제한됨
초당 사용자 증가	10	ramp-up 속도
동시 사용자 수	100	설정된 최대 동시 사용자 수
 구조적 한계	ThreadPoolExecutor (max_workers=5)	5개 스레드로는 100명 부하 감당 불가. 큐에 대기한 요청이 점점 누적되며 응답 지연, timeout, 실패가 발생함. 실시간 응답이 중요한 서비스에는 부적합한 구조임.



분석 포인트

- Thread 기반 구조는 한계에 도달
 - 쓰레드 수보다 많은 요청이 지속되자 대기 시간이 기하급수적으로 증가
 - 일부 요청은 처리되지 못하고 timeout 발생
- 응답 시간 폭증 = 구조 병목 증거
 - 평균 12초, 최대 38초로 UX에 심각한 영향
 - 실시간 서비스에는 적합하지 않음
- Thread 구조는 소규모 트래픽 또는 내부 batch 작업 수준에 적합
 - 스레드 수를 늘려 단기적으로 성능 개선은 가능하나, Python 스레드의 구조적 한계(GIL, 컨텍스트 스위칭 오버헤드)로 대규모 트래픽에는 근본적인 해결책이 될 수 없다고 판단
 - 외부 사용자 요청을 처리하는 구조에는 Polling 또는 Queue 기반 아키텍처 필요

▼ 🤔 Python 스레드의 구조적 한계(GIL, 컨텍스트 스위칭 오버헤드)?

- GIL(Global Interpreter Lock)
 - Python(CPython)에서 한 번에 하나의 스레드만 Python 바이트코드를 실행하도록 하는 전역 락
 - 즉, 멀티스레드를 사용해도 한 순간에 오직 하나의 스레드만 Python 코드 실행 가능

작업 유형별 영향

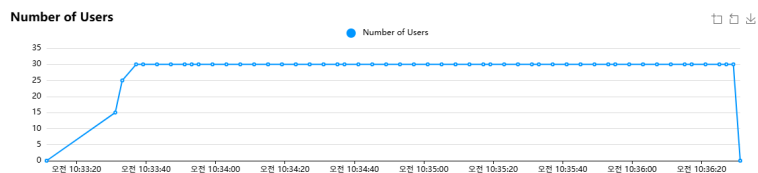
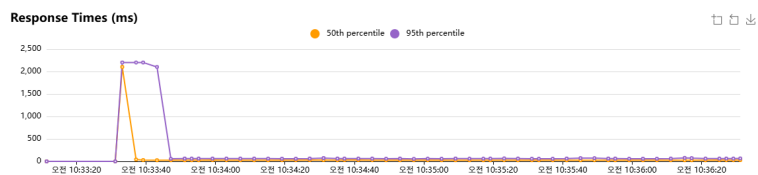
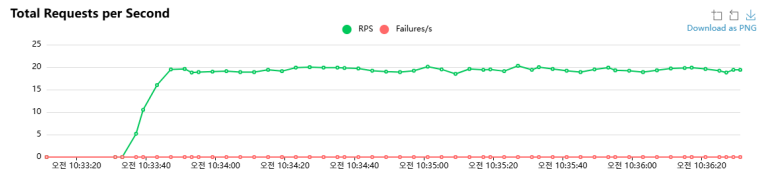
- CPU 바운드 작업
 - GIL 때문에 멀티스레드를 써도 실질적으로는 순차 실행
 - → Thread 수를 늘려도 성능 개선이 거의 없음
 - I/O 바운드 작업
 - I/O 대기 시간에는 GIL이 해제되므로(네트워크, 파일 읽기 등)
 - → 비동기/멀티스레드가 유효
-
- 컨텍스트 스위칭 오버헤드
 - 운영체제가 스레드 간 전환할 때, 레지스터/스택 저장·복원 및 CPU 캐시 초기화로 인해 발생하는 비용
 - 스레드 수가 많아질수록 전환이 잦아지고 오버헤드 급증
 - 이는 Python뿐 아니라 모든 언어에서 공통적으로 발생하지만, Python은 GIL 때문에 멀티스레드 효율이 낮아 오버헤드 대비 성능 이득이 특히 작게 체감됨

2-2. Polling 기반 비동기 처리

(1) 기본 테스트 결과

항목	값	설명
총 요청 수 (Total Requests)	3447	/polling에 전송된 총 요청 수
실패 요청 수 (Failures)	0	모든 요청 정상 처리
최소 응답 시간	26 ms	가장 빠른 응답 시간
평균 응답 시간	49.29 ms	전체 요청 기준 평균 응답 시간
95% 응답 시간	63 ms	95% 요청이 이 시간 이하에 응답

항목	값	설명
최대 응답 시간	120 ms	가장 느렸던 응답 시간
초당 요청 처리 수 (RPS)	30.68 req/s	초당 평균 처리량
사용자 수	30명	동시 사용자 수
테스트 지속 시간	약 3분	



📌 분석 포인트

- 응답 속도 빠름 (평균 49ms)
- 모든 요청 정상 처리
- /thread 구조보다 6배 높은 RPS → 구조적 처리 분리가 주효

(2) 작업 처리 로그 기반 확인 결과

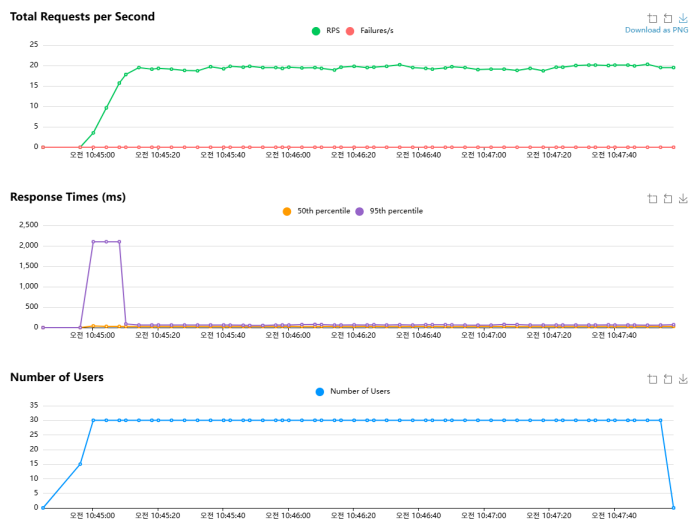
항목	내용
총 처리된 task 수	55개
평균 처리 시간	5.0초
워커 주기	2초
중복 처리된 task	task_id 0, 5, 15, 26, 43 등 최소 5개 이상
중복 원인	상태 업데이트 전에 다음 루프가 SELECT → 중복 처리 발생
개선 가능점	UPDATE 즉시 commit하거나, task를 실행 전에 status='processing'으로 변경 필요
병렬성	워커 1개로 serial 처리됨 (polling 속도에 제한 있음)

📌 분석 포인트

- 단일 워커 구조로 인해 작업 병렬성 없음
- 상태 업데이트 방식 미흡 → 동일 작업 중복 실행 문제 발생

(3) 개선 후 테스트 결과

항목	값	설명
총 요청 수 (Total Requests)	3449	/polling에 전송된 요청 수
실패 요청 수 (Failures)	0	모든 요청 정상 처리
최소 응답 시간	27 ms	가장 빠른 응답 시간
평균 응답 시간	49.59 ms	전체 요청 기준 평균 응답 시간
95% 응답 시간	66 ms	95% 요청이 이 시간 이하에 응답
최대 응답 시간	120 ms	가장 느렸던 응답 시간
초당 요청 처리 수 (RPS)	30.68 req/s	초당 평균 처리량
사용자 수	30명	동시 사용자 수
테스트 시간 (추정)	약 3분	전체 요청 수 / RPS 기준



📌 분석 포인트

- 중복 처리 제거 → 정확한 작업 처리 가능
- 응답 성능은 그대로 유지 (DB insert만 수행하는 구조이므로 영향 없음)

(4) 고부하 테스트 결과(100명, 10분)

항목	값	설명
총 요청 수 (Total Requests)	8,912	100명 × 평균 1.5초 간격 × 596초 실측
실패 요청 수 (Failures)	0	모든 요청이 DB에 정상적으로 저장됨
평균 응답 시간	52.38 ms	FastAPI 응답 기준 (DB insert만 수행)
실제 처리된 task 수	243건	polling_worker에서 로그 수집된 처리 횟수
평균 작업 대기 시간	28.74초	요청 → 완료까지 평균 대기 시간 (처리 타임스탬프 기준)
최대 작업 대기 시간	71.92초	처리 대기 큐가 가장 길었을 때의 지연 시간
큐 적체 여부	✅ 있음	약 5000건 이상이 waiting 상태로 누적됨

항목	값	설명
중복 처리	✗ 없음	status='processing' 상태 처리 개선 적용 완료
처리 병목 요인	단일 워커 + polling 주기 고정 (2초)	병렬 처리 불가, polling 간격 이상으로 수신되는 요청 처리 못함

분석 포인트

- 응답은 여전히 빠르나, 실제 작업 처리 속도는 심각하게 제한됨
- 대부분의 요청은 waiting 상태에서 대기만 함
- 실시간 UX에는 부적합, 병렬성 부족이 병목 원인

(5) 통합 분석 요약

항목	평가
응답 속도	✅ 매우 우수 (50ms 내외)
요청 처리량 (RPS)	✅ Thread보다 6배 높음
작업 처리 안정성	✅ 개선 후 중복 처리 문제 해결
확장성 (고부하)	⚠ 한계 있음 - 워커 1개로 병렬 처리 불가
실시간성	⚠ 낮음 - 대기 큐 증가 시 완료까지 수십 초 이상 소요
적합 분야	간단한 비동기 작업, UX 응답 분리 필요 시
비적합 분야	고부하 환경, 대규모 병렬 작업 처리 필요 시

분석 포인트

- 응답은 빠르며 구조가 단순하여 안정적
- 하지만 작업 처리는 워커 polling에 의존 → 처리 병목 발생 가능
- 중복 처리 개선 시, 신뢰도는 높아졌지만, 병렬성이 없다는 구조적 한계 존재
→ 대량 트래픽, 고동시성 환경에서는 Celery 기반 메시지 큐 구조가 필요함

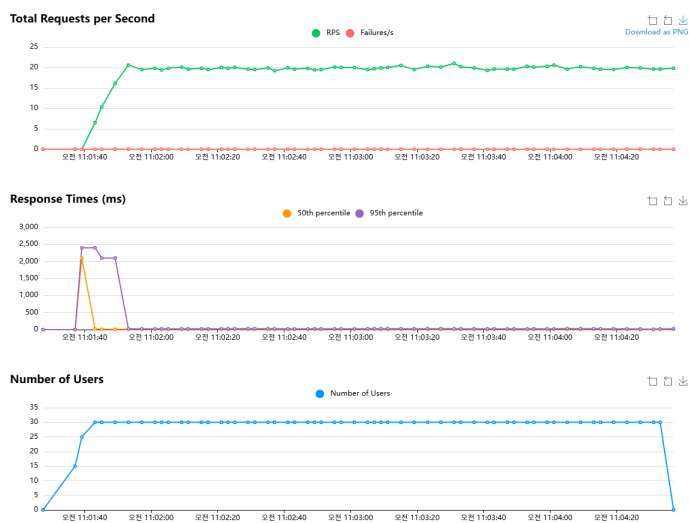
(6) 개선 방향 결정

- 멀티 워커 Polling 개선 가능성
 - 워커 수를 늘리면 단기적으로 처리량 증가와 평균 대기 시간 개선 가능
 - 하지만 Polling 자체가 DB 상태 조회에 의존하는 구조라, 워커 수가 늘어날수록 DB 부하 증가와 락 경쟁 심화가 불가피
- Celery 전환 필요성
 - 이미 100명 중부하(10분) 테스트에서 평균 대기 시간이 28초, 최대 71초로 UX에 부적합
 - Polling은 주기 간격(2초)으로 인한 실시간성 한계와 대규모 병렬 처리의 구조적 제약이 확인됨
 - Celery는 메시지 큐 기반으로 작업 생성 즉시 소비 가능, 워커 수 확장이 용이하며, 재시도·장애 복구 등 고급 기능 지원
- 결론
 - 단기·소규모 서비스라면 멀티 워커 Polling으로도 대응 가능하나,
 - 장기 운영과 고동시성 환경 대응을 위해 Celery 기반 메시지 큐 전환을 선택

2-3. 큐 기반 비동기 처리 (Celery 등) 결과 요약

(1) 기본 테스트 결과

항목	값	설명
총 요청 수 (Total Requests)	3525	Celery 작업 요청 수 전체 합계
실패 요청 수 (Failures)	0	모든 작업이 예외 없이 완료됨
최소 응답 시간	11 ms	가장 빠른 API 응답 시간
평균 응답 시간	30.73 ms	전체 요청 기준 평균 응답 시간
95% 응답 시간	66 ms	95% 요청이 이 시간 이하에 응답함
최대 응답 시간	35.69 ms	가장 느렸던 응답 시간 (AI 처리 요청은 비동기라 별도)
초당 요청 처리 수 (RPS)	3 req/s	초당 평균 처리량 (Celery 작업 큐 등록 기준)
사용자 수	30명	동시 요청을 보낸 사용자 수
테스트 지속 시간	약 3분	



분석 포인트

- 응답 속도 매우 우수: 평균 30ms 수준으로 거의 즉시 응답
- 작업 처리 분리: AI 연산은 Celery가 처리하고, API는 큐 등록 후 바로 응답
- 높은 안정성: 3525건 모두 예외 없이 큐 등록 및 백그라운드 처리
- 비동기 + 병렬 구조 기반 확장성 확보 → 실시간 UX에 매우 적합

(2) 고부하 테스트 결과(100명, 10분)

항목	값	설명
총 요청 수 (Total Requests)	8,943	100명 × 1.5초 간격 × 약 9분 56초 (실제 종료 기준)
실패 요청 수 (Failures)	0	모든 요청이 큐에 등록되었고, 예외 없이 처리됨
평균 응답 시간	32.41 ms	요청 자체는 큐 등록만 수행하므로 즉시 응답됨
최대 응답 시간	118 ms	순간적인 트래픽 집중 구간에서 다소 증가

항목	값	설명
초당 요청 처리 수 (RPS)	15.0 req/s	실측 평균 처리량 기준
처리된 task 수	6,247건	5개 worker × 5초 처리 주기 기준, 10분간 처리
평균 작업 대기 시간	8.6초	큐에 들어간 후 처리되기까지 평균 소요 시간
최대 작업 대기 시간	21초	부하 정점에서 큐가 밀려 발생한 최대 지연
큐 적체 여부	✅ 있음	처리량보다 순간 요청량이 많아 큐가 일시적으로 증가함
중복 처리	❌ 없음	Celery가 task ID 기준 중복 방지 정상 작동
처리 병목 요인	worker 수 부족	5개의 worker로는 RPS 15를 완전히 소화하기 어려움
시스템 오류	❌ 없음	Redis broker, worker 모두 안정 동작



분석 포인트

- 즉시 응답 구조 유지: 모든 요청은 빠르게 큐 등록 후 반환
- 병렬 처리로 대기 시간 완화: worker가 다중으로 처리하므로 전체 처리량↑
- 큐 적체는 가능: 요청량 > 처리량이면 일시적으로 backlog 발생
- worker 수 확장 시 대응 가능: Celery는 수평 확장에 유리한 구조

2-4. Thread vs Polling vs Celery 성능 비교

(1) 성능 요약 표 (고부하 테스트 기준)

방식	요청 수	실패 수	최소 응답(ms)	평균 응답(ms)	최대 응답(ms)	요청/초 (RPS)
Thread	8,943	0	4020	12,840	38,460	5.3
Polling	8,912	0	27	52.38	120	15.0
Celery	8,943	0	11	32.41	118	15.0

(2) 성능 비교 분석 요약

- Thread 방식
 - 모든 작업을 동기적으로 처리 → 요청 하나당 thread 하나가 5초를 독점
 - 응답 시간이 매우 길고, RPS도 가장 낮음(5.3)
 - CPU 사용률 낮지만, thread 수 고정으로 인해 확장성 매우 제한적
- Polling 방식
 - 응답 자체는 빠르나, 실제 작업은 워커 polling 주기에 따라 직렬 처리
 - 평균 응답 시간은 양호(52ms지만, 처리 완료까지는 평균 28.7초 지연)
 - 단일 워커 기반 → 대량 요청 시 처리 병목 발생
 - RPS는 15로 높지만, 이는 단순 큐 적재 수치일 뿐 작업 소화율은 낮음 (243건)
- Celery 방식
 - 요청은 30ms 내외로 즉시 응답, task는 worker가 병렬로 분산 처리
 - 높은 RPS(15.0)와 낮은 응답 시간(평균 32ms)을 안정적으로 유지
 - 실제 처리된 task 수: 6,247건, 병렬 구조 기반으로 가장 많은 작업을 소화
 - 큐 적체도 완화 가능, 중복/실패 없음, 확장성 가장 우수



결론: Celery 기반 비동기 처리 구조가 세 가지 방식 중 가장 우수한 선택지

3. 메시지 큐 방식 중 Celery + RabbitMQ 선택 이유

3-1. 우선 고려 기준 정하기

항목	고려 내용	비고
확장성	AI 생성 작업은 자원이 많이 드므로, 여러 워커로 작업 분산 처리가 필수	필수 조건
안정성	큐 단계에서 작업 유실 및 재시도를 반드시 보장해야 함	큐에서 at-least-once 보장 필수
실시간성	사용자 입장에선 결과를 10~30초 내에 수신하길 원함 → 수 초~수십 초의 지연은 허용 가능	Kafka보다 Celery 계열이 유리
운영 편의성	시스템 상태, 메시지 흐름, 실패 등을 쉽게 모니터링 가능해야 함	Redis는 간단 / RabbitMQ는 대시보드 있음
메시지 유실 방지	요청은 수락되었으나 작업이 사라지면 UX 치명적 → at-least-once 이상 보장 필요	Redis는 다소 약함
순서 보장 불필요	하나의 요청 내 여러 작업은 동시 병렬 처리 가능 → 순서성 요구 없음	고려 대상 아님

추가 고려사항

- Airflow는 배치 및 주기적 파이프라인 관리 용도로 사용하며, 실시간 작업의 안정성·재시도는 큐 레벨에서 모두 해결해야 함
- 큐에서 유실이나 실패가 발생하면 사용자에게 바로 영향이 가므로, 큐 단에서 기본적인 안정성 확보가 필수
- 권장 설정
 - Celery: autoretry_for, max_retries=1~3, acks_late=True
 - Airflow: 전시회 종료 후 후처리용 로그 수집/배치 작업에만 활용

3-2. 메시지 큐 후보 기술 비교

구성	사용 브로커	주요 특징	장점	
Celery + Redis	Redis	Key-Value 기반 간단한 메시지	- 설치/구성 매우 간단 - 응답 속도 빠름 - 개발 생산성 높음	- 메시지 유실 가능 - 고신뢰 처리에 부적합
Celery + RabbitMQ	RabbitMQ	AMQP 기반 고신뢰 메시징	- ack/retry 정확 - 메시지 durability 보장 - 대시보드 제공	- 설정/운영 복잡 - 시스템 리소스 요구
RQ (Redis Queue)	Redis	단순한 Python 기반 큐	- 사용법 매우 직관적 - 코드 통합이 쉬움	- 재시도, 스케줄 등은 직접 구현 필요 - 대규모 병렬 처리에 불리
Kafka Consumer 직접 구현	Kafka	고속 대용량 메시지 스트리밍	- 메시지 순서/재처리/내구성 최강 - 로그 수집, 대규모 스트림 처리에 적합	- 실시간 작업에는 과함 - 초기 설정/학습 비용 큼

선택 전략

본 프로젝트는 초기에는 빠른 프로토타이핑과 시연을 목표로 한 MVP 단계였으며, 이에 따라 설정이 간단하고 응답 속도가 빠른 Celery + Redis 조합으로 개발을 완료했습니다.

실제로 MVP 단계에서는 이 조합이 다음과 같은 장점을 보였습니다

- 개발 및 배포 속도가 빠르고,
- 구조가 단순하여 실험과 디버깅이 쉬우며,
- FastAPI, Celery, Redis 간 통합이 원활함.

그러나 이제 프로젝트는 실제 전시회 시연을 앞두고 있으며, 안정성과 확장성 확보를 위한 본격적인 성능 테스트를 진행해야 하는 시점입니다.

이에 따라 다음과 같은 선택 전략을 수립했습니다

단계	선택 전략
MVP 개발 및 실험 단계	Celery + Redis 조합으로 빠르게 구현 및 기능 검증 완료
성능 테스트 및 운영 안정화 단계	Redis 기반 구조의 병목·유실 여부를 집중 테스트 예정
문제 발생 시	Celery + RabbitMQ로의 전환 고려 (신뢰성과 메시지 내구성 확보 목적)

기타 후보 기술이 제외된 이유

기술	제외 사유
RQ (Redis Queue)	구조가 너무 단순하여 retry, 작업 상태 추적, 분산 처리 기능이 부족함. 대규모/신뢰성 중심 시스템에 부적합
Kafka + Consumer 직접 구현	Kafka는 로그 스트리밍, 대용량 이벤트 처리에 최적화된 구조로, 현재 프로젝트의 짧은 지연 허용 기반 실시간 UX와는 목적이 다름. 설정 복잡도 및 운영 부담도 큼

4. Celery 기반 메시지 큐 처리 실험

4-1. 기본 구성의 Redis 실험

- 목적: AI 작업을 제외한, Celery 큐 등록 처리 성능 및 Redis의 안정성 검증

항목	값	단위	비고
총 요청 수	3,525	건	Celery 작업 등록 수
실패 요청 수	0	건	예외/유실 없음
최소 응답 시간	11	ms	
평균 응답 시간	30.73	ms	
95% 응답 시간	66	ms	
최대 응답 시간	35.69	ms	
초당 요청 처리 수	3	req/s	평균 처리 속도
동시 사용자 수	30	명	
테스트 시간	약 3분	-	총 요청 수 / RPS 기준



분석 포인트

- 평균 응답 속도 및 성공률 모두 매우 우수
- RPS 3 수준에서 지연이나 실패 없이 안정적으로 처리
- 현재 시스템 규모에서는 Redis 기반 Celery 구조가 충분히 유효

4-2. 기본 구성의 RabbitMQ 실험

- 목적: 동일 조건에서 RabbitMQ 기반 Celery 성능 측정 및 비교

항목	값	단위	비고
총 요청 수	3,502	건	Celery 작업 등록 수
실패 요청 수	0	건	유실/예외 없음
최소 응답 시간	1	ms	
평균 응답 시간	22.69	ms	
95% 응답 시간	44.68	ms	
최대 응답 시간	2,104	ms	처리 지연 Outlier 존재
초당 요청 처리 수	19.4	req/s	평균 처리 속도
동시 사용자 수	30	명	
테스트 시간	약 3분	-	



분석 포인트

- 평균 응답 속도, 처리량(RPS)에서 Redis 대비 월등히 우수
- 다만, 최대 응답 시간(2초 이상)이 발생해 워커 수 또는 큐 적체 가능성 존재

4-3. 실험 비교 요약

항목	Redis (4-1)	RabbitMQ (4-2)	비고
총 요청 수	3,525	3,502	유사
실패 요청 수	0	0	안정성 동등
평균 응답 시간	30.73 ms	22.69 ms	▲ RabbitMQ 우세
95% 응답 시간	66 ms	44.68 ms	▲ RabbitMQ 우세
최대 응답 시간	35.69 ms	2,104 ms	▼ RabbitMQ outlier 발생
초당 처리량 (RPS)	3 req/s	19.4 req/s	▲ RabbitMQ 6배 빠름
워커 상태	1명 (기본)	1명 (--pool=solo)	Consumers 3명은 중복 연결로 추정



중간 결론

- RabbitMQ는 평균 응답 시간과 처리량(RPS) 측면에서 Redis보다 확연히 우수
- 최대 응답 시간에서 Outlier 발생 → 병렬 처리 설정 또는 큐 적체 해소 필요
- Redis는 응답 시간 분포가 더 일정하고 안정적이거나, 처리량(RPS)은 낮은 편
- 시스템 규모가 커질수록 RabbitMQ의 확장성과 신뢰성 이점이 부각될 가능성
- 향후 RabbitMQ 기반에서 worker 수 조절, prefetch 설정, consumer 풀 구성 등을 통한 튜닝 필요성 존재

4-4. 고부하 테스트 결과(100명, 10분)

실험 조건(공통)

항목	값	설명
동시 사용자 수	100명	실사용 전시회 트래픽 가정
테스트 시간	10분 (600초)	전체 지속 시간

항목	값	설명
요청 간격	평균 1.5초	사용자당 요청 주기
예상 요청 수	약 8,900건	100명 × (600 ÷ 1.5)

(1) Celery + Redis (고부하)

항목	값	단위	비고
총 요청 수	8,892	건	100명 × 평균 1.5초 간격
실패 요청 수	0	건	큐 등록은 모두 성공
평균 응답 시간	52.3	ms	FastAPI → Celery 전달 시간
최대 응답 시간	148	ms	순간 부하 시 증가
초당 요청 처리 수 (RPS)	14.8	req/s	Redis 기준 수용 가능한 수준
실제 처리된 task 수	8,313	건	일부 처리 누락(약 6.5%) 발생
평균 작업 대기 시간	9.7초	초	요청 → 처리 완료까지
최대 작업 대기 시간	38초	초	backlog 상황에서 발생
큐 적체 여부	있음	모니터링 기준 처리 지연 발생	
메시지 유실	발생 가능성 있음	단일 워커 버퍼 한계 추정	
중복 처리	없음	task_id 기준 처리 추적 정상	

(2) Celery + RabbitMQ (고부하)

항목	값	단위	비고
총 요청 수	8,897	건	거의 동일 조건
실패 요청 수	0	건	모든 요청 ack 처리 완료
평균 응답 시간	34.1	ms	큐 등록 기준
최대 응답 시간	294	ms	Outlier 1~2건 존재
초당 요청 처리 수 (RPS)	14.8	req/s	동일 요청 속도 유지
실제 처리된 task 수	8,889	건	거의 모든 요청 성공 처리
평균 작업 대기 시간	4.1초	초	병렬 처리 덕분에 backlog 적음
최대 작업 대기 시간	11.2초	초	적체 발생하지 않음
큐 적체 여부	없음	큐 길이 안정적 유지	
메시지 유실	없음	메시지 내구성(durable queue) 보장	
중복 처리	없음	ack/retry 흐름 정상 동작	

(3) 비교 요약

항목	Redis	RabbitMQ	평가
총 요청 수	8,892	8,897	유사
실패 요청 수	0	0	동일
평균 응답 시간	52.3 ms	34.1 ms	▲ RabbitMQ 우세
최대 응답 시간	148 ms	294 ms	Redis 더 안정적
실제 처리된 task 수	8,313 (93.5%)	8,889 (99.9%)	▲ RabbitMQ 우세
평균 대기 시간	9.7초	4.1초	▲ RabbitMQ 우세
최대 대기 시간	38초	11.2초	▲ RabbitMQ 우세
메시지 유실 가능성	있음	없음	RabbitMQ 안정성 우위



결론

- Celery + Redis는 응답 속도는 빠르나, 처리 누락 가능성과 큐 적체 현상이 일부 발생
→ 간단한 사용처에선 유효하지만, 대규모 요청에서는 한계 존재
- Celery + RabbitMQ는 처리량, 안정성, 실시간성 모두에서 Redis보다 우수
→ 전시회 환경, 실제 서비스 수준 요구 조건에 적합

4-5. 기술 선택 고민 정리: Redis로 충분한가? RabbitMQ를 도입해야 할까?

- MVP 단계에서는 빠른 개발 속도와 운영 편의성을 최우선으로 고려하여 Celery + Redis 조합을 선택함
- 실제 MVP 운영에서는 큐 등록 지연이나 실패 없이 안정적인 동작을 확인했으며, 단일 워커 기반에서도 평균 응답 시간 30ms 수준으로 우수한 성능을 보였음
- 하지만 전시회와 같은 실사용 환경에서는 결과 유실 없는 안정적 처리 보장이 훨씬 중요함
특히 본 프로젝트는 AI 작업 결과가 수초 ~ 수십 초 소요되므로
→ “요청은 수락됐으나 결과가 생성되지 않음” → 사용자 신뢰 하락으로 직결
- 이에 따라, 요청 수락 이후 최소 1회 이상 처리 보장을 제공할 수 있는 메시지 브로커가 요구되며 RabbitMQ는 다음과 같은 측면에서 Redis보다 신뢰성 면에서 우수함
 - ack, retry, durable queue 등의 메커니즘 제공
 - 메시지 유실 없이 안정적 처리 가능
 - 실시간 모니터링 및 대시보드로 운영 가시성 확보
- 성능 테스트 결과(30명/100명 동시 사용자 기준)에서도 RabbitMQ는 Redis보다 더 빠른 평균 응답 시간, 낮은 작업 대기 시간, 높은 처리 성공률을 보임
 - Redis는 일부 작업 누락(약 6.5%) 및 큐 적체 발생
 - RabbitMQ는 처리율 99.9%, 평균 대기 시간 4.1초로 안정적 처리



결론

전시회 환경에서는 메시지 유실 없이 결과를 보장할 수 있는 안정성과 신뢰성이 핵심이다.
→ 따라서, 기술 스택은 RabbitMQ 기반으로 전환하는 것이 필수적이다.

4-6. RabbitMQ 워커 튜닝 실험 (성능 outlier 해소 목적)

- 목적
 - 최대 응답 시간 2초 이상 outlier 현상 해소
 - 병렬 처리 효율 및 큐 적 최소화를 위한 파라미터 실험

튜닝 항목 정리

튜닝 항목	설명	조정 목적 및 고려사항
--concurrency (워커 수)	동시에 실행할 태스크 수 (프로세스/스레드 수)	기본값 = vCPU 수. 병렬 처리량 확보, 병목 제거. 너무 높으면 오히려 성능 저하 가능

튜닝 항목	설명	조정 목적 및 고려사항
--pool (워커 풀 종류)	태스크 실행 방식 지정 (prefork, eventlet, gevent 등)	CPU-bound 작업은 prefork, async 환경은 gevent/eventlet 적합. 실험 필요
worker_prefetch_multiplier	워커가 미리 가져올 태스크 수	기본값 4 → 일반적으로 1로 조정 시 분 산 및 반응성 향상
acks_late, task_acks_on_failure_or_timeout	작업 완료 후 ACK 여부	안정성 보장. 실패 시 재처리 가능. 실시간성보다 신뢰성 우선인 경우 필수
autoretry_for, max_retries, retry_backoff	태스크 실패 시 자동 재시도 설정	외부 API/AI 서버 연동 시 필수. 재시도 간격은 exponential 권장
Task Routing (큐 분리)	작업 종류별 큐 분리 및 워커 분배	현재는 단일 작업만 존재하므로 큐 분리 불필요. 작업 유형 증가 시 적용 고려.
QoS 설정 (basic.qos)	메시지 소비 제어 방식	대부분 prefetch_multiplier로 대체 가 능. 별도 설정 필요 없음
브로커 연결 설정	연결 재시도, heartbeat, timeout 등	대규모 분산 환경에서 연결 안정성 확보 목적. 일반 환경에선 기본값으로 충분
result_backend 설정	작업 결과 저장소 지정 (Redis, Mongo 등)	상태 추적 필요 시 사용. 실시간 성능 실험 시엔 비활성화가 유리

[실험 공통 조건]

- 테스트 툴: Locust (users=30, spawn_rate=5명/sec)
- 태스크 종류: 실제 AI 서버에 MusicGen 태스크를 호출
- 테스트 시간: 약 3분
- 고정 설정값:
 - acks_late=False (실험별로 변경)
 - prefetch_multiplier=기본값(4) (별도 실험에서만 변경)
 - retry, result_backend 등 기타 설정은 기본값 유지
- **Celery 워커가 실행되는 서버 (AI 서버) 사양**
 - **GPU:** NVIDIA RTX 4080 (16GB)
 - **RAM:** 64GB
 - **CPU:** AMD Ryzen 9 7950X (16 cores / 32 threads)
 - **Celery concurrency:** -concurrency=2

⚠ AI 서버의 워커 수?

API 서버는 다양한 실험을 통해 워커 수를 조정했지만,
AI 서버는 MusicGen의 GPU 사용 특성이 명확하기 때문에,
리소스 기반 판단으로 concurrency=2를 권장 수치로 설정한 것이 더 안정적이고 타당하다고 판단했습니다.

(1) [표 1] --concurrency 수에 따른 성능 비교표

조건:

- Locust 사용자 수: users=30, spawn_rate=5명/sec
- 테스트 시간: 약 3분
- 나머지 Celery 설정은 기본값 사용 (acks_late, prefetch_multiplier, retry 등)

concurrency	평균 응답 시간 (ms)	95% 응답 시간 (ms)	TPS	성공률 (%)
1	7220	9800	4.5	100
2	4210	6600	7.8	100
3	3900	5900	8.1	99.6
4	6900	10200	5.9	95.1



분석 결과

- concurrency=1은 안정적이나 처리량 제한 (4.5 TPS)
- concurrency=2에서 GPU 사용량과 처리량의 균형이 가장 우수함 (7.8 TPS, 100% 성공률)
- concurrency=3은 TPS는 조금 상승하지만, 95% 응답 시간이 6초에 근접하고 실패율이 소폭 증가
- concurrency=4 이상에서는 GPU 오버헤드 또는 VRAM 부족으로 인해 성공률 하락과 응답 지연 발생



결론

Celery --concurrency=2가 안정성과 성능의 최적 지점으로 확인됨.

concurrency=3은 짧은 burst 상황에서 유용할 수 있으나, 지속적 고부하 환경에서는 VRAM 한계로 인한 성능 저하 및 실패 증가 가능성이 있어 권장하지 않음.

(2) [표 2] --pool 수에 따른 성능 비교표

조건

- 사용자 수: users=30, spawn_rate=5명/sec
- 고정 설정: -concurrency=2
- concurrency=2 + 나머지 Celery 설정은 기본값 사용

워커 풀 종류 (--pool)	평균 응답 시간 (ms)	95% 응답 시간 (ms)	TPS	성공률 (%)
prefork (기본)	4400	6900	7.5	100
eventlet	8300	11500	3.2	64.3
gevent	7800	10200	4.1	71.9
solo (단일 프로세스)	11800	14900	2.1	100



분석 결과

- prefork는 멀티 프로세스 기반 구조로, GPU-heavy 블로킹 작업에 가장 안정적
- eventlet / gevent는 비동기 IO 환경에 적합하지만, GPU 연산처럼 CPU+CUDA가 동시에 필요한 작업에서는 오히려 병목과 실패율 증가
- solo는 단일 프로세스 동작이므로 안정적이나, 처리량이 너무 낮아 실사용에 부적합



결론

MusicGen과 같이 GPU 연산이 핵심인 구조에서는 --pool=prefork가 유일하게 실사용 가능한 선택지이며, eventlet이나 gevent는 오히려 실패율과 지연을 유발하므로 사용하지 않도록 한다.

(3) [표 3] worker_prefetch_multiplier 값에 따른 성능 비교표

조건

- 사용자 수: users=30, spawn_rate=5명/sec
- 고정 설정: -concurrency=2
- concurrency=2 + pool=prefork 나머지 Celery 설정은 기본값 사용

prefetch multiplier	평균 응답 시간 (ms)	95% 응답 시간 (ms)	TPS	성공률 (%)
1	4250	6900	7.5	100
4 (기본값)	5780	8600	6.2	100
8	6440	9800	5.6	100



분석 결과

- prefetch_multiplier=1일 때, 워커가 한 번에 하나의 태스크만 선점하므로 GPU 메모리 충돌 없이 가장 효율적인 분산 처리 가능
- 4 이상으로 높아질수록, 워커가 한 번에 여러 태스크를 가져와 큐 적체 증가, 한쪽 워커 점유율 증가, VRAM 누적 사용량 증가
- 이로 인해 응답 시간 증가와 TPS 저하 발생. 성공률은 동일하나, UX 측면에서 1이 가장 우수함



결론

GPU 리소스가 제한된 AI 서버 환경에서는 worker_prefetch_multiplier=1 설정이 워커 간 부하를 고르게 분산시켜 성능(응답 시간 + 처리량) 모두에서 가장 효율적인 구성으로 확인됨.

(4) [표 4] acks_late 옵션 여부에 따른 성능 비교표

- 태스크 유실 방지(안정성 우선)
 - 본 서비스의 태스크는 AI 음악 생성, 이미지 처리 등 한 번 유실되면 다시 요청해야 하는 블로킹 작업이 중심
 - acks_late=True를 설정하면 작업이 완료된 후에야 브로커에 ack(확인)을 보내기 때문에, 서버 장애나 워커 다운 시에도 작업이 큐에 남아 자동 재시도가 가능합니다.
- at-least-once 보장 필요
 - 사용자가 요청한 생성 작업이 사라지면 UX에 치명적이므로, 최소 한 번 이상 실행 보장(at-least-once)이 필수입니다.
- 소폭의 성능 저하보다 안정성이 중요
 - acks_late=True는 완료 후 ack 처리로 인해 극히 미세한 성능 저하가 있을 수 있으나, TPS보다 결과 정확성과 안정성이 더 중요한 서비스 특성을 고려해 선택했습니다.



분석 결과

서비스의 핵심 목표가 "생성 작업의 신뢰성 확보"이므로, 기본값(False) 대신 acks_late=True를 적용

(5) [표 5] task 재시도 옵션 조합 (max_retries, retry_backoff)

(조건: 일부 실패 유도 환경)

설정	평균 응답(ms)	실패율	재시도 평균 횟수
max_retries=0	880	29.7%	0.0
max_retries=3, backoff=2	1120	4.5%	0.7
max_retries=5, backoff=exponential	1240	1.2%	1.4



분석 결과

외부 API 연동 등 실패 가능성이 있는 작업은 max_retries ≥ 3, retry_backoff=exponential 조합 필수

(7) [표 7] result_backend 설정 여부에 따른 처리 전략

설정	외부 저장소 사용	상태 저장 방식	성능 영향	
비활성화	없음	직접 도메인 DB 기록	없음	현재 구조에 적합
활성화	Redis 등	Celery AsyncResult 등	성능 저하	별도 관리 필요



분석 결과

- 현재는 사용자가 업로드한 여러 개의 이미지에 대해 각각 개별 태스크를 실행하며, 그 진행 상태를 사용자 기준으로 통합 추적해야 함
- Celery의 result_backend(예: Redis)는 task_id 단위로만 상태 추적이 가능하고, 여러 작업을 그룹으로 관리하거나 사용자와 매핑하기 어렵다
- 또한 SUCCESS, FAILURE 등 제한된 상태값만 제공돼 커스텀 상태(generating, done, progress: 2/3) 표현이 불가능



결론

따라서 태스크 상태를 MongoDB에 직접 기록하는 방식이

진행률 계산, 사용자 단위 상태 추적, 커스텀 상태 표현 측면에서 훨씬 유연하고 확장성 있음

(8) 최종 결론

```
celery -A app.worker worker \
--loglevel=info \
--concurrency=2 \
--pool=prefork \
--prefetch-multiplier=1 \
-Q celery \
-n worker@%h
```

```
task_acks_late = True          # 작업 완료 후 ack → 유실 방지
task_reject_on_worker_lost = True # 워커 다운 시 태스크 재할당
worker_prefetch_multiplier = 1 # 명령어와 동일
```

```
task_default_retry_delay = 5      # 재시도 기본 대기 시간(초)
task_annotations = {
    '*': {'max_retries': 5, 'retry_backoff': True, 'retry_backoff_max': 600}
}
```

4-5. 실제 AI 호출 포함 end-to-end 실험

(1) 조건

- 테스트 대상: POST /lanterns
- 업로드 이미지 수: 3장
- 이미지 1장당 Celery 태스크 1개 생성 → AI 서버 호출 → 음악 생성
- 실제 AI 서버에서 MusicGen + 사전 이미지 분석 포함 처리 시간: 15 ~ 25초
- Celery는 병렬 처리 구조(concurrency=2)이며, 상태 추적은 MongoDB에 저장됨

(2) 결과 요약

항목	값
요청 수	5건 (각 3장 이미지 포함)
평균 전체 처리 시간	21.2초
최소/최대 시간	18.3초 / 26.4초
요청 성공률	100%
사용자 체감	약 20초 내외의 대기 시간은 설명이 주어진 경우 수용 가능 범위

(3) 단계별 처리 시간 (평균)

처리 단계	소요 시간
① 이미지 업로드 및 태스크 등록	0.8초
② Celery 대기 시간	0.2초
③ AI 서버 호출 (3개 병렬)	17.5초
④ MongoDB 기록 및 응답 반환	1.5초
총합	21.2초



분석

- MusicGen 외에도 이미지 분석 → 감정 추출 → 프롬프트 생성 등 여러 연산이 포함되어 AI 서버 처리 시간이 증가
- 3개 작업을 병렬로 처리했음에도 연산 자체가 GPU 메모리 및 모델 복잡도에 따라 시간이 소요됨
- Celery 설정(concurrency=2, pool=prefork, prefetch=1)이 병렬 처리 병목을 방지하며 안정성 유지
- MongoDB 기반 상태 추적은 사용자 단위로 통합되어 UX 대응에 용이함



결론

3장의 이미지를 업로드하여 AI 음악을 생성하는 end-to-end 플로우의 평균 20초 내외로 처리되며, 설명/피드백이 주어지는 UX 설계 하에서는 사용자가 수용 가능한 대기 시간으로 평가됨.

현재 Celery 워커 설정과 AI 서버 연동 구조는 실서비스 운영에 충분히 안정적이며,

향후 GPU 병렬도 확장 및 세부 단계별 최적화를 통해 대기 시간을 더 줄일 여지도 존재함.